



Lezione 7



Sviluppo Applicazioni Mobili
V. Gervasi – a.a. 2023/24



Un esempio completo

BMI Calc



- Facciamo un esempio completo
 - La più banale cosa che viene in mente: BMI calc
 - Due campi di testo, in cui l'utente immette altezza e peso
 - Un pulsante calcola, che fa il semplice conto del Body Mass Index
 - Un'area di testo, in cui mostrare il risultato
 - (c'è chi c'ha fatto i soldi... ma il mondo era giovane allora)

Vari stili implementativi



Scrivere a mano un layout XML

Si crea una risorsa `res/layout/nome.xml`, indicando i tag XML e le relative proprietà; il nesting dei tag definisce la struttura di contenimento.



Comporre graficamente un layout

Si disegna la GUI tramite l'editor grafico, in parte per manipolazione diretta di elementi grafici, e in parte attraverso l'impostazione di attributi da palette.



Creare un albero di View e ViewGroup

Si creano “a programma” oggetti di (sottoclassi di) classe View tramite l'operatore **new**; si impostano le proprietà con metodi `set...()` e si costruisce l'albero con `vg.addView(v)`

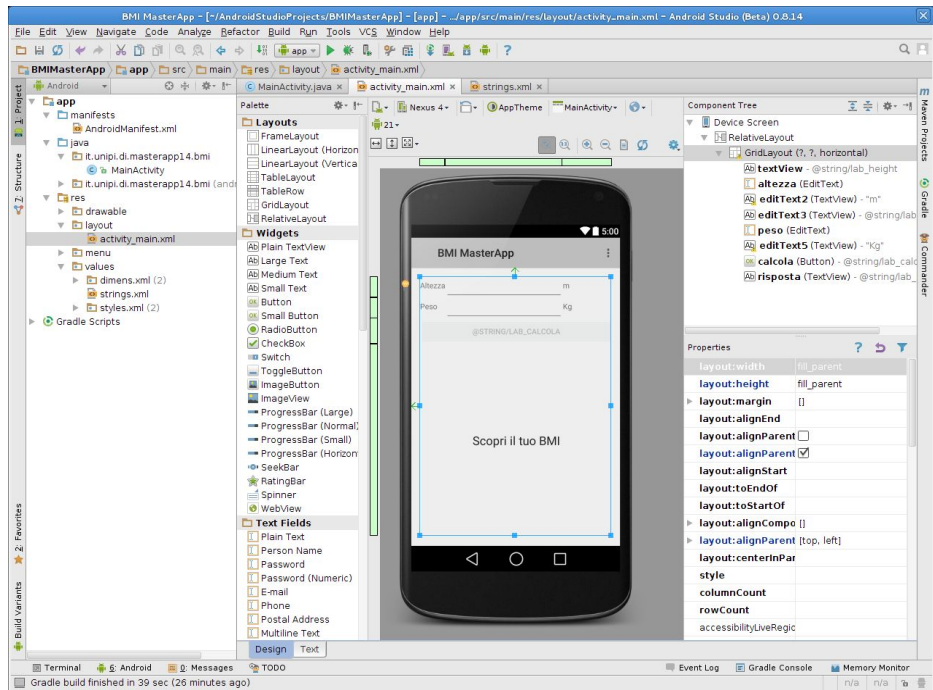


Usare Jetpack Compose

Si usano alcune caratteristiche di Kotlin e un plug-in del compilatore per processare parte del codice “a programma” come una dichiarazione di interfaccia grafica

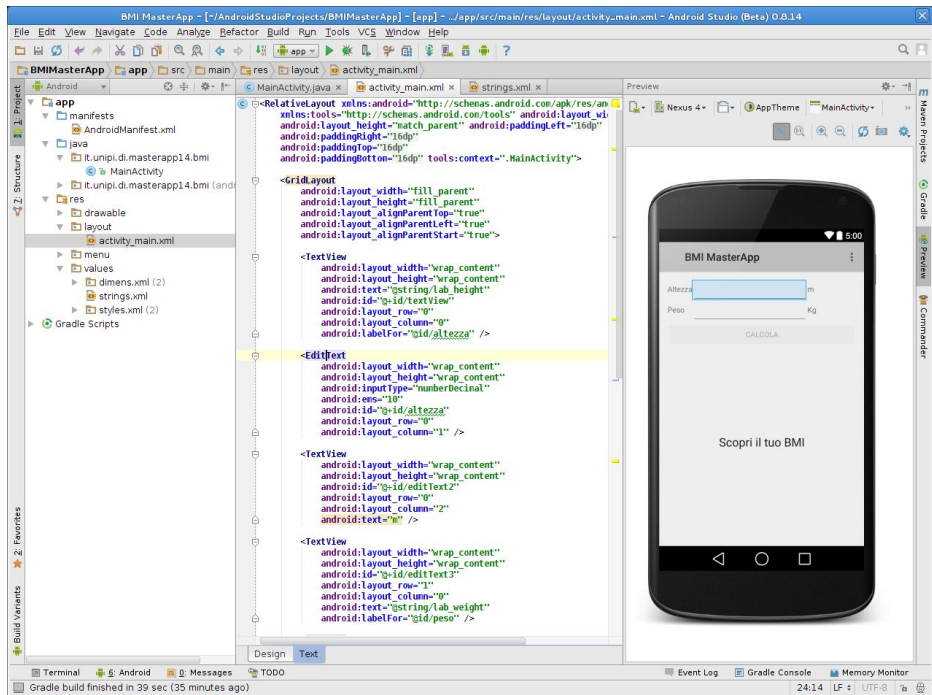


BMI calc – layout



- Ha senso usare l'editor grafico per creare il layout

BMI calc – layout



- Ha senso usare l'editor grafico per creare il layout
- Però lo illustriamo nella forma XML

Schema generale dell'UI (GridLayout)



	0	1	2
0	Altezza		m
1	Peso		kg
2	[Calcola]		
3	Risultato		

res/layout/activity_main.xml



<RelativeLayout

```
xmlns:android="http://schemas.android.com/apk/res/android"
xmlns:tools="http://schemas.android.com/tools"
android:layout_width="match_parent"
android:layout_height="match_parent"
android:paddingLeft="@dimen/activity_horizontal_margin"
android:paddingRight="@dimen/activity_horizontal_margin"
android:paddingTop="@dimen/activity_vertical_margin"
android:paddingBottom="@dimen/activity_vertical_margin"
tools:context=".MainActivity" >
```

xmlns: definisce i namespace XML utilizzati in questo file.

android: dati del framework di Android

tools: dati dei tool (nel nostro caso, Android Studio)

<GridLayout

```
android:layout_width="fill_parent"
android:layout_height="fill_parent"
android:layout_alignParentTop="true"
android:layout_alignParentLeft="true"
android:layout_alignParentStart="true" >
```

Ricordate? È il formato per riferire le risorse all'interno di un file XML:

@tipo/nome

... *contenuti veri* ...

</GridLayout>

</RelativeLayout>

res/layout/activity_main.xml



<TextView

```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:text="@string/lab_height"  
android:id="@+id/textView"  
android:layout_row="0" android:layout_column="0"  
android:labelFor="@id/altezza" />
```

Etichetta per “Altezza”. Notate che il testo della stringa è esternalizzato come risorsa.

<EditText

```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:inputType="numberDecimal"  
android:ems="10"  
android:id="@+id/altezza"  
android:layout_row="0" android:layout_column="1" />
```

Campo di testo editabile, di tipo numerico, per inserire il valore dell'altezza (in metri).

<TextView

```
android:layout_width="wrap_content"  
android:layout_height="wrap_content"  
android:id="@+id/editText2"  
android:layout_row="0" android:layout_column="2"  
android:text="m" />
```

Etichetta per l'unità di misura. Questa volta text **non** è una risorsa: “m” = “metro”, e la nostra formula richiede l'input in metri.



res/layout/activity_main.xml



La seconda riga del layout, per il peso, è del tutto analoga e non la mostriamo.

<Button

```
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:text="@string/lab_calcola"
android:id="@+id/calcola"
android:layout_row="2" android:layout_column="0" android:layout_columnSpan="3"
android:enabled="false" />
```

Pulsante "Calcola" – inizialmente disabilitato

<TextView

```
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:textAppearance="?android:attr/textAppearanceLarge"
android:text="@string/lab_initresult"
android:id="@+id/risposta"
android:layout_row="3" android:layout_column="0" android:layout_columnSpan="3"
android:layout_gravity="fill"
android:gravity="center_vertical|center_horizontal" />
```

Questo è un riferimento a un *attributo* di uno *stile* di sistema

?[package:][tipo]nome

Campo di testo per la risposta.
Occupi tutto lo spazio rimanente, con il testo centrato.

res/values/[...].xml



```
<?xml version="1.0" encoding="utf-8"?>
<resources>
  <string name="app_name">BMI MasterApp</string>
  <string name="hello_world">Hello world!</string>
  <string name="action_settings">Settings</string>
  <string name="lab_height">Altezza</string>
  <string name="lab_weight">Peso</string>
  <string name="lab_initresult">Scopri il tuo BMI</string>
  <string name="lab_calcola">Calcola</string>
</resources>
```

res/values/strings.xml

(ovviamente, dovrebbe essere
res/values-it/strings.xml)

res/values/dimens.xml

```
<resources>
  <!-- Default screen margins, per the Android Design guidelines. -->
  <dimen name="activity_horizontal_margin">16dp</dimen>
  <dimen name="activity_vertical_margin">16dp</dimen>
</resources>
```

AndroidManifest.xml



- Una volta create le risorse (layout e valori) per la nostra App, ci rimangono da creare i componenti (codice)
- In un caso così semplice, basta un solo componente
 - La singola Activity che gestisce la schermata di input e output
- Definiamola subito nel manifesto:

AndroidManifest.xml



```
<?xml version="1.0" encoding="utf-8"?>
<manifest
    xmlns:android="http://schemas.android.com/apk/res/android"
    package="it.unipi.di.sam.bmi" >
    <application
        android:allowBackup="true"
        android:icon="@drawable/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme" >
        <activity
            android:name="it.unipi.di.sam.bmi.MainActivity"
            android:label="@string/app_name" >
            <intent-filter>
                <action android:name="android.intent.action.MAIN" />
                <category android:name="android.intent.category.LAUNCHER" />
            </intent-filter>
        </activity>
    </application>
</manifest>
```



MainActivity.java



```
package it.unipi.di.sam.bmi;  
import ...
```

```
public class MainActivity extends Activity implements OnClickListener {  
    private Button calcola;  
    private EditText altezza, peso;  
    private TextView risposta;  
    @Override  
    protected void onCreate(Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        setContentView(R.layout.activity_main);  
  
        calcola=(Button)findViewById(R.id.calcola);  
        altezza=(EditText)findViewById(R.id.altezza);  
        peso=(EditText)findViewById(R.id.peso);  
        risposta=(TextView)findViewById(R.id.risposta);  
  
        calcola.setOnClickListener(this);  
        calcola.setEnabled(true);  
    }  
}
```

// dovremmo farlo solo dopo l'input negli altri campi...

MainActivity.java



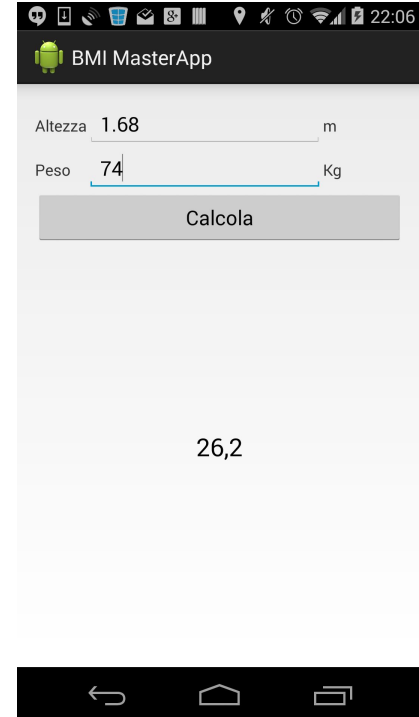
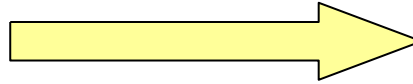
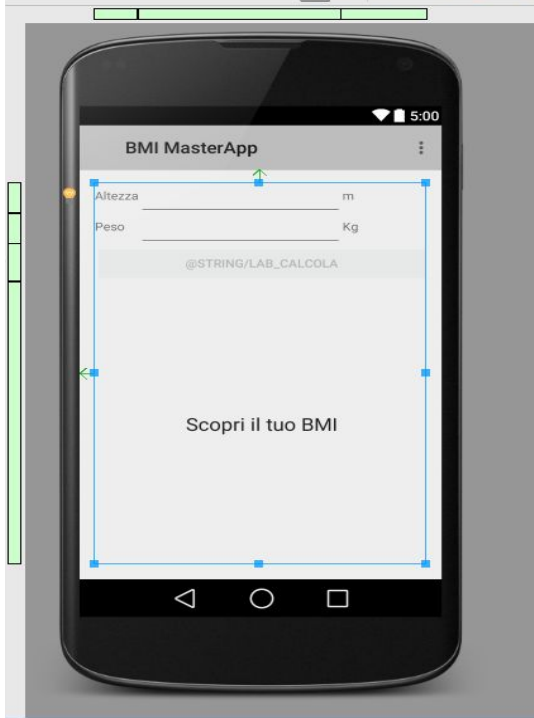
@Override

```
public void onClick(View v) {  
    if (v==calcola) {  
        try {  
            double a = Double.parseDouble(altezza.getText().toString());  
            double p = Double.parseDouble(peso.getText().toString());  
            double bmi = p / (a * a);  
            risposta.setText(String.format("%.1f", bmi));  
        } catch (Exception e) {  
            risposta.setText("?");  
        }  
    }  
}
```

Per testare:

- telefonino abilitato da sviluppatore
- telefonino in modalità Debug USB abilitata
- telefonino collegato via USB al PC
- Ctrl-Alt-F10 su Android Studio

BMI calc – finale



Jetpack Compose



Jetpack è una raccolta di librerie che offrono funzionalità di tipo **avanzato** che riguardano vari aspetti della programmazione Android

In particolare: *architettura generale* delle App, gestione dei *dati* e della relativa *sincronizzazione*, gestione della *user interface*, gestione della *concorrenza*, delle *comunicazioni*, ...

In generale, Jetpack implementa pattern comuni e best practices emerse negli anni

Jetpack Compose è la libreria Jetpack dedicata a costruire GUI

Basata su **Kotlin**

Fa largo uso di caratteristiche specifiche di Kotlin:

- *annotazioni*
- *funzioni costruttore*
- *parametri con nome*

La GUI viene definita all'interno del codice
Supporto parziale da parte dell'editor grafico

Struttura della UI espressa attraverso funzioni @Composable

MainActivity.kt



```
Column {
    Row {
        Text("Altezza")
        TextField(value = altezza, {altezza=it},
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal))
        Text("m")
    }
    Row {
        Text("Peso")
        TextField(value = peso, {peso=it},
            keyboardOptions = KeyboardOptions(keyboardType = KeyboardType.Decimal))
        Text("kg")
    }
    Button(
        { val a=altezza.toDouble()
          val p=peso.toDouble()
          bmi=String.format("%.1f",p/(a*a))      },
        content={Text("Calcola")}
    )
    Text(bmi)
}
```

BMIC calc – finale

- Brutto come la fame
 - non abbiamo indicato alcun parametro di layout, allineamenti, distanze, ecc.
 - sarebbero tutti parametri aggiuntivi delle funzioni costruttore!
- Però funziona!
 - devo dimagrire almeno 5 kg...

